

A VAE and a DDPM on CelebA

Two generative models of 64×64 faces, built from primitive PyTorch layers

The task

Learn a distribution over CelebA faces and sample new ones.

VAE — compress an image to a 128-number code, decode codes back to pixels.

DDPM — learn to reverse a process that turns an image into noise.

202,599 images · 64×64 · scored with the same FID and IS pipeline.

72.06

VAE — FID

35 min training

9.63

DDPM — FID

7 h 13 min training

Preprocessing

1. Crop the central 178×178 square — the extra height is background.
 2. Resize to 64×64 .
 3. Random horizontal flip — training split only.
 4. Scale pixels to $[-1, 1]$.
-

Why $[-1, 1]$: the VAE decoder ends in $\tanh$, so data and output space match.

The DDPM mixes images with unit Gaussian noise centered around 0.

The generative story

1. Draw a latent code

$$z \sim \mathcal{N}(0, I)$$

2. Decode it into pixels

$$x \sim p_{\theta}(x | z)$$

$p(z | x)$ is intractable, so a second network approximates it:

$$q_{\phi}(z | x) = \mathcal{N}(\mu(x), \text{diag } \sigma^2(x))$$

The objective

$\log p(x)$ is intractable — the ELBO bounds it:

$$\log p(x) \geq \mathbb{E}_{z \sim q}[\log p(x | z)] - \text{KL}(q(z | x) \parallel p(z))$$

Negating gives two terms with distinct jobs:

$$\mathcal{L} = \text{reconstruction} + \text{KL}$$

Reconstruction keeps the decoder honest (resemble the input image as close as possible).

KL pulls the encoder's output onto the prior — which is what makes $N(0, I)$ a valid place to sample from.

The two terms

Reconstruction

$$\mathcal{L}_{\text{rec}} = \sum_{i=1}^{12288} (x_i - \hat{x}_i)^2$$

KL, in closed form

$$\text{KL} = \frac{1}{2} \sum_j (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2)$$

The reconstruction is just squared error, while the KL closed form rewards the zero mean and unit variance.

Reparameterization

Sampling is not differentiable — so rewrite z as a deterministic function of the parameters plus parameter-independent noise:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

$$\frac{\partial z}{\partial \mu} = 1, \quad \frac{\partial z}{\partial \sigma} = \varepsilon$$

Gradients now reach the encoder. Without this the VAE cannot be trained at all.

One training step

1. Encode the batch $\rightarrow \mu, \log \sigma^2$
2. Sample $z = \mu + \sigma \odot \epsilon$
3. Decode $\rightarrow \hat{x}$
4. Loss = summed squared error + KL
5. Backpropagate — through the decoder, and through μ and σ into the encoder

Architecture

Encoder	$4 \times (\text{Conv } k=4 \ s=2 \ p=1 \rightarrow \text{BatchNorm} \rightarrow \text{LeakyReLU})$	$64 \rightarrow 512 \text{ ch}, 64^2 \rightarrow 4^2$
Heads	two Linear layers	μ and $\log \sigma^2$, 128-d
Decoder	$\text{Linear} \rightarrow 4 \times \text{ConvTranspose} \rightarrow 3 \times 3 \text{ conv} \rightarrow \tanh$	$4^2 \rightarrow 64^2, 3 \text{ ch out}$
Size	8.73M parameters	4.86M encoder + 3.88M decoder

k=4, s=2, p=1 halves H exactly, and its transpose doubles it exactly.

A final **3×3 conv** cleans up transposed-convolution checkerboard artefacts.

Why VAE samples are blurry

Per-pixel squared error is the log-likelihood of a Gaussian decoder.

When several plausible images fit a code equally well, the loss is minimised by their **average**.

The average of many sharp faces is a smooth face.

This follows from the objective, so no amount of training fixes it.

It is the main thing on which diffusion wins.

The forward process

A fixed process adds a little Gaussian noise at each of $T = 1000$ steps:

$$q(x_t | x_{t-1}) = \mathcal{N}(\sqrt{1 - \beta_t} x_{t-1}, \beta_t I)$$

$$\alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{s \leq t} \alpha_s$$

β grows linearly from $1e-4$ to 0.02 . After 1000 steps the image is indistinguishable from noise.

A network learns to undo one step; generation starts from noise and runs backwards.

The formula that makes it practical in training

Composing t Gaussian steps gives another Gaussian:

$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$$

so in code, one multiply-add:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon$$

Training at timestep 700 costs **one** operation, not 700 simulated steps.

This is why DDPM can be in parallel, while sampling is expensive and sequential.

The training objective

The variational bound simplifies to a regression: given a noisy image, predict the noise.

$$\mathcal{L} = \mathbb{E}_{x_0, t, \varepsilon} \|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2$$

One training step

1. Draw a clean image x_0 from the batch
2. Draw a timestep $t \sim \text{Uniform}(0, 999)$ and noise $\epsilon \sim \mathcal{N}(0, I)$
3. Form x_t in closed form — one multiply-add
4. Predict the noise $\hat{\epsilon}$ with the U-Net
5. MSE, backpropagate, then update the EMA weights

The network

Same shape in and out — an image in, a noise map out.

Skip connections carry high-frequency detail around the bottleneck. Noise is entirely high-frequency, so a plain autoencoder would destroy exactly what we ask it to predict.

Self-attention at 16×16 and below — convolutions are local, so nothing else forces two eyes to match. Cost is $(H \cdot W)^2$, affordable only at low resolution.

GroupNorm, not BatchNorm — sampling runs at tiny batch sizes.

The timestep enters every residual block as a per-channel bias:

$$\text{emb}(t)_i = \sin(t / 10000^{i/h}), \quad \text{emb}(t)_{h+i} = \cos(t / 10000^{i/h})$$

Sampling — ancestral, 1000 steps

Predict the noise → convert to an estimate of x_0 → clamp → step to the posterior mean → add noise.

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$\mu_t = \frac{\beta_t \sqrt{\bar{\alpha}_{t-1}}}{1 - \bar{\alpha}_t} \hat{x}_0 + \frac{(1 - \bar{\alpha}_{t-1}) \sqrt{\alpha_t}}{1 - \bar{\alpha}_t} x_t$$

1. $x_T \sim \mathcal{N}(0, I)$
2. For $t = T - 1, \dots, 0$:
 1. $\hat{\varepsilon} = \varepsilon_\theta(x_t, t)$ — one full forward pass of the U-Net
 2. $\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\varepsilon}}{\sqrt{\bar{\alpha}_t}}$, clamped to $[-1, 1]$
 3. $\mu = c_1(t) \hat{x}_0 + c_2(t) x_t$
 4. $x_{t-1} = \mu + \sigma_t z$, with $z \sim \mathcal{N}(0, I)$ and $\sigma_t = \exp(\frac{1}{2} \log \tilde{\beta}_t)$
 5. At $t = 0$, return μ without adding noise
3. Return $x_0 \in [-1, 1]$

Sampling — DDIM, 50–100 steps

The objective only constrains the marginals $q(x_{\square} \mid x_0)$. Many reverse processes share them — including a deterministic one that skips timesteps.

$$x_{\text{prev}} = \sqrt{\bar{\alpha}_{\text{prev}}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{\text{prev}} - \sigma^2} \varepsilon_{\theta} + \sigma z$$

$$\sigma = \eta \sqrt{\frac{1 - \bar{\alpha}_{\text{prev}}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{\text{prev}}}}$$

50 steps instead of 1000 is a 20× speedup for a small quality cost — and the only reason scoring 10,000 images was feasible.

The weight EMA

Diffusion quality is unusually sensitive to weight noise, so we sample from a smoothed copy:

$$\text{shadow} \leftarrow d \cdot \text{shadow} + (1 - d) \cdot \text{current}$$

But the shadow starts as a copy of the random initialisation, and with a fixed $d = 0.9999$ it stays there:

updates	still random init
1,000	90%
11,000	33%
30,000	5%

The fix — ramp the decay in from near zero:

$$d_{\text{eff}} = \min \left(d, \frac{1 + \text{step}}{10 + \text{step}} \right)$$

At step 0 that is 0.1, so real weights dominate immediately.

FID — lower is better

Summarise each image set as one 2048-d Gaussian over InceptionV3 features, then measure the distance:

$$\text{FID} = \|\mu_1 - \mu_2\|^2 + \text{Tr}(\Sigma_1 + \Sigma_2 - 2(\Sigma_1 \Sigma_2)^{1/2})$$

First term: how far apart the two *average images* sit. Second: how differently the sets are spread.

So FID responds to **realism and diversity** — one perfect face every time scores terribly.

Inception Score — and why it is weak here

$$\text{IS} = \exp(\mathbb{E}_x[\text{KL}(p(y|x}) \parallel p(y))])$$

Large when samples are confident **and** the set is diverse. Uses no labels, and never looks at the real images.

But ImageNet has no *face* class. Over 500 real CelebA images, Inception answers:

Windsor tie	83 / 500
maillot	65 / 500
jersey	35 / 500
brassiere	33 / 500
tench (a fish)	24 / 500

Real CelebA scores IS = 3.07 ± 0.09 — don't know what it means here? Ceiling maybe?

Results

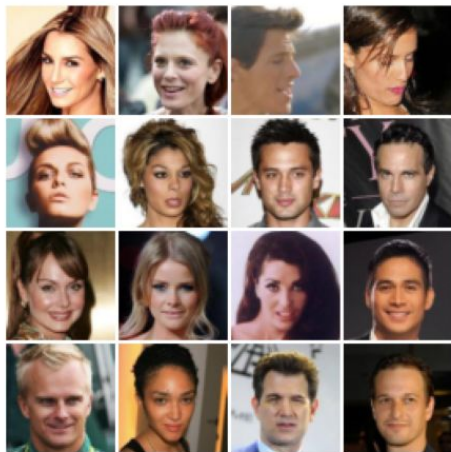
Model	FID ↓	IS ↑	Params	Training
VAE	72.06	1.95 ± 0.03	8.73M	25 ep · 35 min
DDPM (DDIM, 100 steps)	9.63	2.57 ± 0.04	17.22M	20 ep · 7 h 13 min
real CelebA	0	3.07 ± 0.09	—	—

RTX 3060 Laptop GPU (6 GB) · 10,000 generated images scored against 10,000 real.

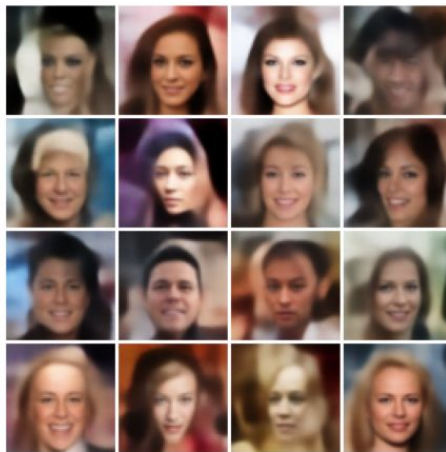
Real · VAE · DDPM

Uncurated samples at 64x64, identical display scale

Real CelebA



VAE (1 forward pass)



DDPM (DDIM, 100 steps)

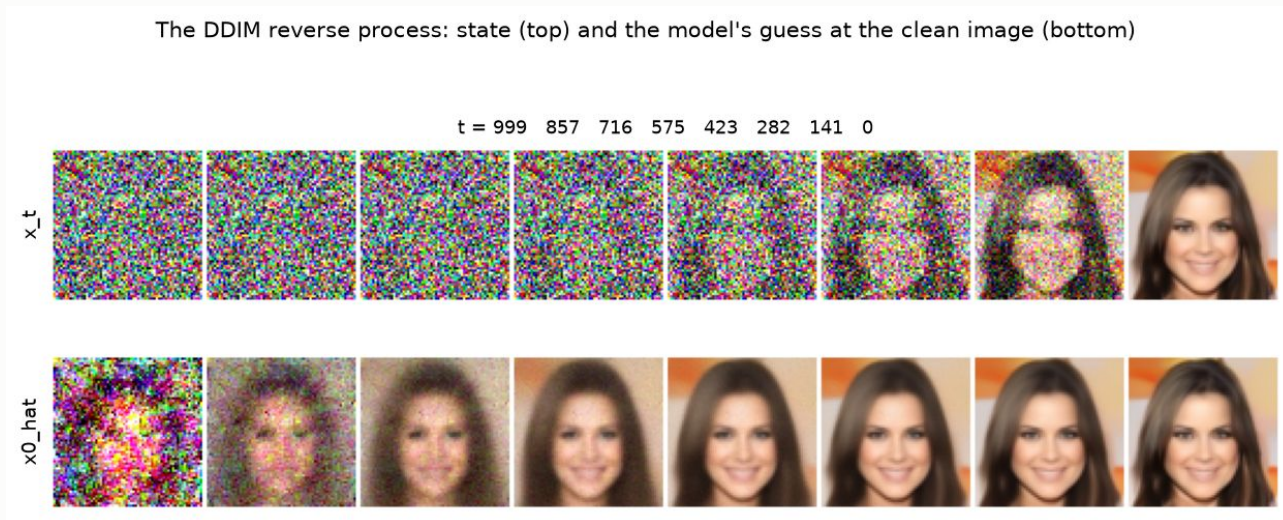


What the 128-d code keeps



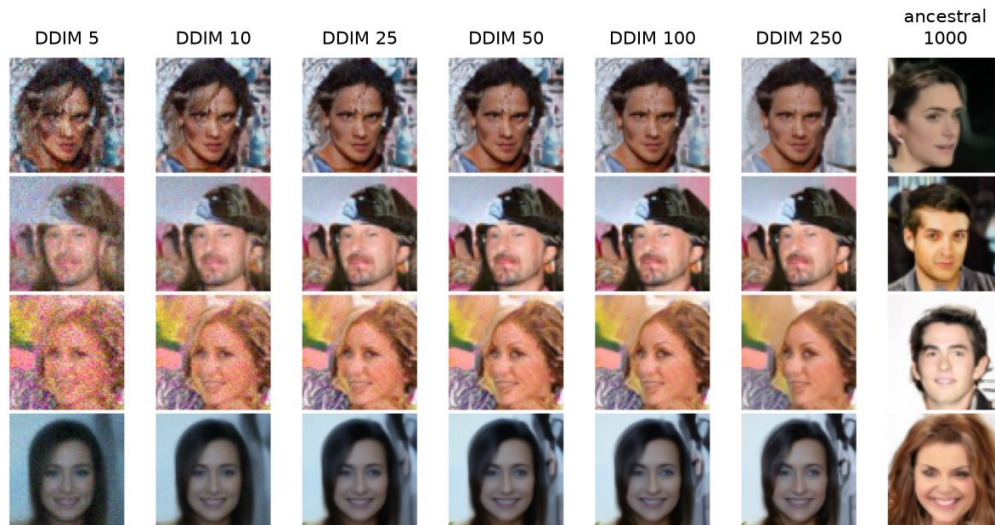
Top four rows real, bottom four reconstructed. Kept: identity, pose, gaze, hair, lighting. Lost: everything that occurs once — text on a sign, glasses, a smile's shape.

What the reverse process does



How few steps you can get away with

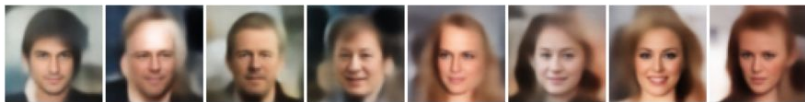
Same x_T , increasing sampling budget



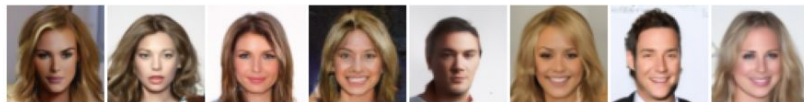
The two models fail in opposite directions

Ranked by cosine similarity to the closest real image (Inception features)

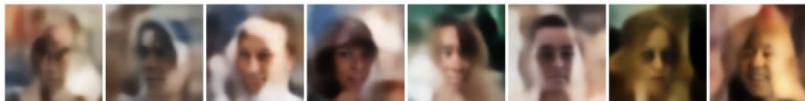
VAE - most typical



DDPM - most typical



VAE - least typical



DDPM - least typical



The VAE's worst samples are boring — off-centre, dark, blurrier than its median, but always face-shaped. Failure mode: *absence of detail*.

The DDPM's worst samples are wrong — melted features, a mouth in the wrong place. Confidently rendered nonsense. Failure mode: *commitment to the wrong answer*.

Thank you

Abdelrahman Elaraby